Team Rayquaza: Transformer Fleet Health & Replacement Planning

HCLTech Future Ready AI Engineer Hackathon · DS/ML edition · 11 Oct 2026 · design brief

WHAT WE UNDERSTOOD

A utility has about 400 transformers and money to fix only a few. We predict each one's **chance of failing in the next 12 months**, compute **risk = probability × impact** (hospital 5, industry 3, urban 2, rural 1, × customers served), and choose which to **replace or inspect within the budget**, explaining every choice. A transformer with no oil test is never called "healthy".

ARCHITECTURE: MEDALLION (RAW → CLEANED → BUSINESS-READY)

4 CSVs **Source** synthetic, seeded → raw, as-is **Bronze** never edited → validated **Silver** clean, typed, deduped → modelled **Gold** star schema + snapshots → decision **Risk + plan** model, knapsack → serve **API + UI** FastAPI, Streamlit

LAYER	WHAT HAPPENS (PROCESS)	WHY IT EXISTS
Bronze	Each CSV is loaded **exactly as received** into `bronze_oil_tests`, `bronze_operation`, `bronze_customers`, `bronze_costs`, with `_ingested_at` and `_source` added. The late data drop (`/ingest`) also lands here first.	The **replay tape**: if a later step has a bug, we fix it and re-run from bronze without re-fetching anything. It is also the audit trail of what arrived and when.
Silver	**Validate** (HCL checks as `assert`s: no future dates, gases > 0, every unit has a customer type, row counts match the data card) → **clean**: standardise IDs (`"tr-0045 "` → `TR-0045`), remove duplicate assets and rows, fix gas **unit jumps** (ppb logged as ppm, ×1000), cast types. Gaps stay **NULL and flagged**, never silently filled.	**One place** where data quality is enforced. Every later step can trust silver. Bad data fails loudly here instead of quietly corrupting the model.
Gold	**Star schema:** `dim_transformer` (customer type, customers, impact weight, region, size class) · `dim_date` (calendar / quarter) · `fact_oil_test` (one row per transformer per test) · `fact_operation` (one row per transformer per month) · `fact_snapshot`: one row per transformer per quarter, with features built **only from data before that date** (latest test via `ROW_NUMBER()`, change between tests via `LAG()`, 12-month load and temperature aggregates).	Shaped for **reading**, not storing. Dimensions are loaded once and read constantly. The snapshot table is the single, leakage-safe input to the model and the API.
Decision	Model scores each snapshot → `fact_risk` (probability, range, impact, risk) → **knapsack** picks actions within the budget → compared with an age-based plan.	Numbers come from code, not from text. Every run is stored, so we can show what changed after the late data drop.

Grain of the core table: one row of `fact_snapshot` = **one transformer at one quarterly snapshot date**. Features use only data from *before* that date; the label is "failed in the following 12 months". This is how leakage is prevented by design.

HOW ONE RUN WORKS

1. `python src/pipeline.py`: one command, one status line per stage.
2. Load → bronze → validate + clean → silver → build gold.
3. Score every transformer → risk → budget plan.
4. **Re-runnable:** silver and gold are rebuilt in full, so running twice gives identical results.
5. **Atomic:** gold is written in one transaction; a failed load rolls back.
6. **Late drop (13:45):** new oil tests + new budget → `/ingest` → bronze → re-run → show what changed.

KEY DESIGN DECISIONS

- **Medallion over one script:** raw data is kept, so any step can be replayed and audited.
- **Star schema over 3NF in gold:** fewer joins for reading; we accept some redundancy in dimensions.
- **Quarterly snapshots:** monthly gives near-duplicate rows (tests every 6–12 months); yearly gives too few.
- **Full refresh over incremental:** about 15k rows rebuild in seconds; incremental would need a watermark and adds failure modes.
- **SQLite:** no server, nothing to break on venue wifi; window functions do the as-of joins.
- **SCD Type 1** on `dim_transformer`; Type 2 noted as a known limitation.

MODEL LAYER: KEPT SIMPLE

Compare simple **baselines** (age-only, a rule-based health score) against **logistic regression**. A more complex model is kept only if it clearly beats them. Probabilities are **calibrated** so "20%" means about 20%. Testing is honest: train on earlier periods, test on later ones and on transformers the model has never seen.

DATA: WE GENERATE ALL 4 FILES

- 400 transformers, 36 months (Oct 2023 – Sep 2026), about 38% never oil-tested.
- Failures follow a documented rule (age, overload, gas fault) plus randomness.
- All 6 HCL edge cases planted on purpose, listed in the data card.